# Einführung in die Git-Dateiversionierung

git

## Contents

1 Ziel ..... 1
2 Installation ..... 2
3 Basis Operationen ..... 5
4 Branch und Merge ..... 10
5 Gitflow ..... 11
6 GIT Befehle ..... 13
Bibliographie ..... 15

## 1 Ziel

In diesem Labor lernen wir die Grundprinzipen von der Versionskontrolle `git` kennen [1]. Es ist vorteilhaft vor oder während dem Labor den Theorieteil durchzuarbeiten.
Zu Beginn werden wir Git auf Ihrem Rechner installieren und konfigurieren siehe Abschnitt 2. Sowie Konten auf Github [2] sowie Hevs Gitlab [3] erstellen.
Im Abschnitt 3 lernen wir die Basis Operationen kennen um mit Git arbeiten zu können.
Das erstelle Repository wird danach auf GitHub veröffentlicht.
Die erweiterten Funktionen `branch` sowie `merge` werden in einem Beispiel probiert im Abschnitt 4.
Im Abschnitt 5 arbeiten wir alle gemeinsam nach dem Gitflow Prinzip.

# 2 Installation

Der erste Schritt ist die Installation von Git sowie Sublimemerge.

## 2.1 `git`

Du kannst die neueste Version über die offizielle Website https://git-scm.com/ [1] herunterladen. Git ist für Linux, Mac und Windows verfügbar. Für dieses Labor wird git ≥ 2.27 benötigt.

### 2.1.1 Kommandozeile

Starte "Git Bash". Dies ist ein Unix/Linux-ähnlicher Befehlseditor, der es ermöglicht, Git-Befehle im Konsolenmodus auszuführen. Diese Schnittstelle werden wir in diesem Labor verwenden.

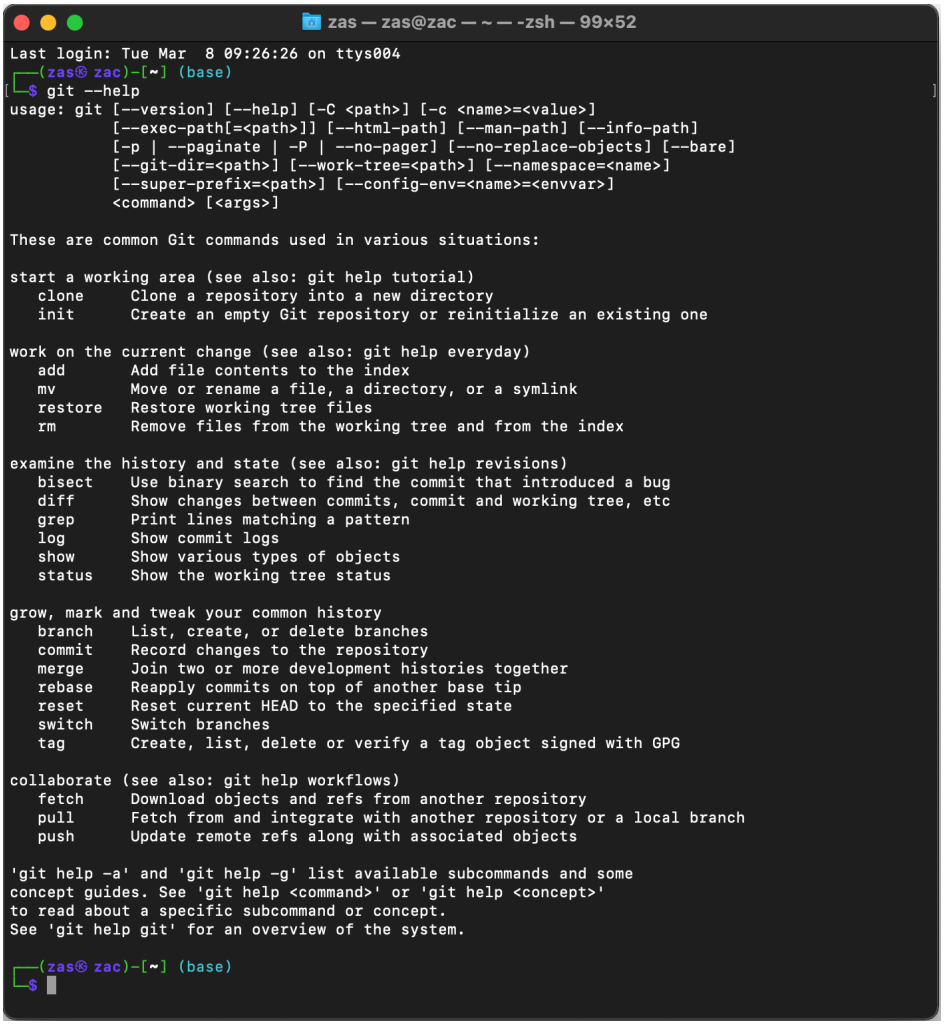


Abbildung 1: git Terminal

> ⚠ Beachten Sie, dass Sie bei allen Befehlen in Git Bash Hilfe erhalten, indem Sie nach dem Befehl `--help` einfügen.

### 2.1.2 Globale Konfiguration

Eine Vielzahl von Einstellungen kann in Git konfiguriert werden. Es ist möglich, die Einstellungen global auf deinem Computer (Flag `--global`) oder nur für ein bestimmtes Repository zu ändern. Wir werden nun die Minimalkonfiguration durchführen. Verwende die folgenden Befehle, um deine Identität in Git global auf dem System einzustellen. Verwende deinen Namen und deine E-Mail-Adresse. Diese Informationen sind öffentlich sichtbar, um deine Arbeit (deine Commits) zu identifizieren.

```
git config --global user.name "Firstname Lastname"
git config --global user.email first.last@email.ch
```

Sie können die Konfiguration mit dem folgenden Befehl überprüfen:

```
git config --list
```

Sie können auch eine bestimmte Einstellung überprüfen:

```
git config user.name
```

## 2.2 Sublime Merge

Besuchen Sie die Webseite https://www.sublimemerge.com and laden sowie installieren Sie das Tool Sublime Merge herunter [4].

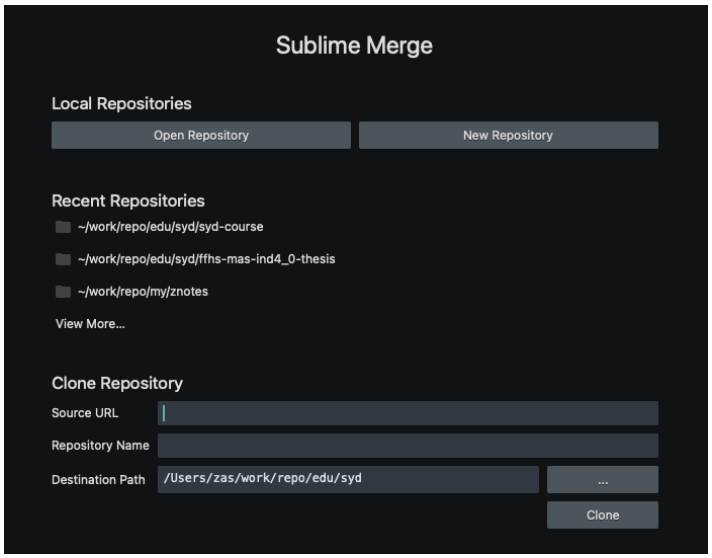


Abbildung 2: Sublime Merge GUI

## 2.3 Online-Konten

### 2.3.1 Gitlab

Besuchen Sie wie Webseite https://gitlab.hevs.ch und loggen Sie sich mit Ihrem Schulkonto ein (SwitchEDU-ID).

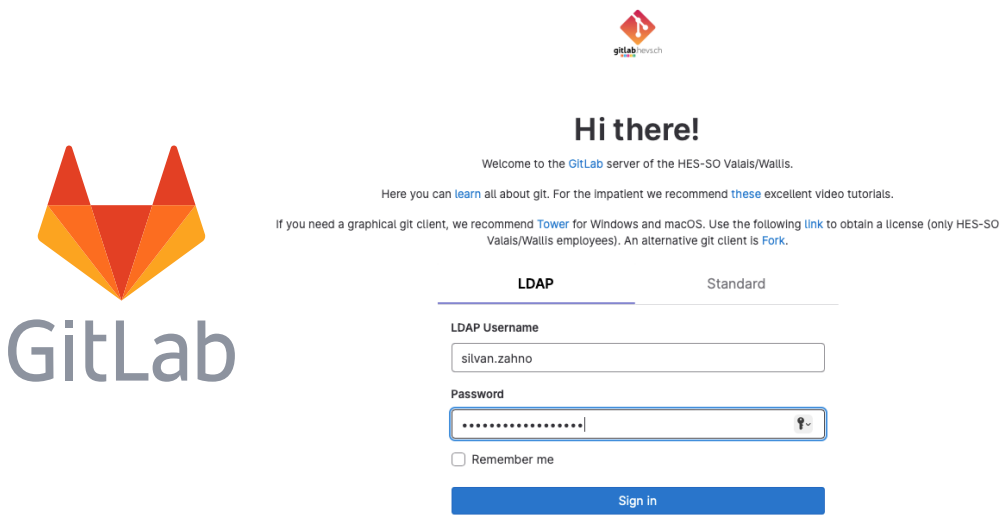


Abbildung 3: Gitlab Anmeldung

### 2.3.2 Github

Besuchen Sie wie Webseite https://github.com und erstellen Sie ein Konto und loggen Sie sich ein.

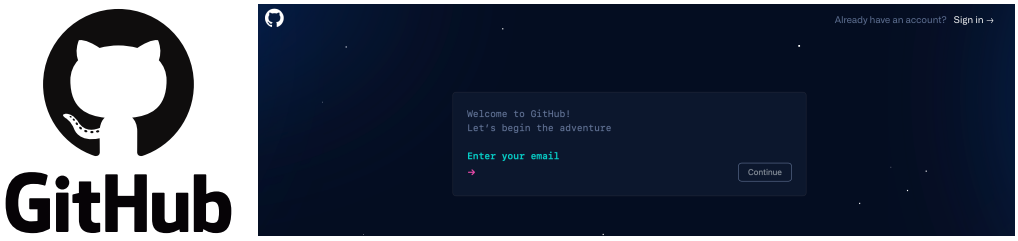


Abbildung 4: GitHub Anmeldung

# 3 Basis Operationen

## 3.1 Erstellen eines git Repositories

Erstellen Sie mithilfe der Git Bash Konsole ein leeres Verzeichnis auf Ihrem Computer, z.B. `C:\temp\gitRepo` oder `~/tmp/gitRepo`. Dieses Verzeichnis wird Ihr Git-Repository sein.
Du kannst die Unix/Linux-Befehle `ls` (list files and directories), `cd` (change directory), `pwd` (print current working directory) und `mkdir` (make directory) verwenden, um dieses Verzeichnis zu erstellen.

```
mkdir -p c:\temp\gitRepo
cd c:\temp\gitRepo
```

## 3.2 Initialisieren

Sobald du in diesem Verzeichnis bist, initialisiere es als Git Repo mit dem Befehl:

```
git init
```

Was hat sich in dem Verzeichnis geändert, nachdem du es als Git-Repo initialisiert hast?

> Schreiben Sie die Antwort auf!

## 3.3 Status abfragen

Erhalten Sie Informationen über Ihr Repo mit den folgenden Befehlen:

```
git status
git log --oneline
```

## 3.4 Datei hinzufügen

Erstellen Sie nun eine leere Datei mit dem Namen `README.md` im Hauptverzeichnis Ihres Repos. Verwenden Sie den vorherigen Befehl, um erneut die Informationen über Ihr Repo abzurufen. Was hat sich geändert?

> Schreiben Sie die Antwort auf!

Ihr lokales Git-Repo besteht aus drei Bereichen, die von git gepflegt werden:

- Das Working directory ist ein Verzeichnis, das die aktuelle Version deiner Dateien enthält (in den Augen deines Betriebssystems ein normales Dateiverzeichnis)
- Stage enthält die Änderungen, die in den nächsten Commit aufgenommen werden sollen;

- Der Head zeigt auf den Ort im Git-Repo-Baum, an dem der nächste Commit durchgeführt werden soll.

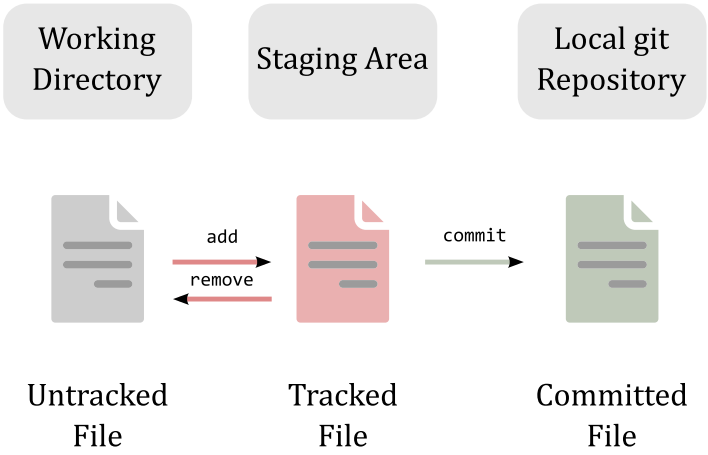


Abbildung 5: Arten von lokalen Git Operationen

Ein einfaches Git-Repo, das aus fünf Commits besteht, kann folgendermassen dargestellt werden. Die Position `Head` ist ein Verweis auf einen Commit, der den aktuellen Status/die aktuelle Ansicht des Repos darstellt, hier die letzte Änderung.

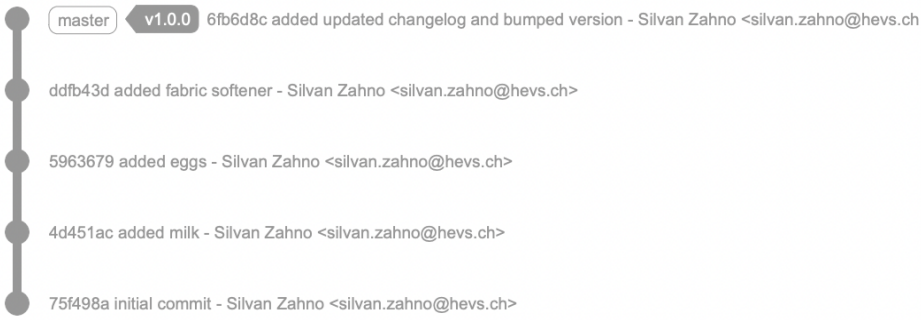


Abbildung 6: Fünf commits auf dem lokalen Repo, jeder commit besitzt seine eigene identifikation

## 3.5 Datei in Repo aufnehmen

Fügen Sie die zuvor erstellte Datei `README.md` zur Stage hinzu, mit dem Befehl:

```
git add README.md
```

Rufen Sie erneut die Informationen über Ihr Repo ab, was stellen Sie fest?

> Schreiben Sie die Antwort auf!

Bearbeiten Sie die Datei `README.md` mit einem texteditor und fügen Sie den folgenden Text ein (Markdown-Syntax):

```
# Title of my readme
Text and more text, followed by a small list :
  * Item 1
  * Item 2
  * Item 3

And finally a little code:
```sh
$ cd myDir
$ git init
$ git status
$ ls -al
```
```

Erhalte erneut die Informationen über dein Repo, was stellst du fest?

Schreiben Sie die Antwort auf!

## 3.6 Neue Änderungen hinzufügen

Fügen Sie die neueste Version der Datei `README.md` zum Stage hinzu.

```
git add README.md
```

## 3.7 Commit ausführen

Führen Sie nun einen Commit mit folgendem Befehl durch:

```
git commit -m "Initial commit. Add README file."
```

Die Option `-m` erlaubt es, die Nachricht des Commits direkt anzugeben. Diese Nachricht muss selbsterklärend sein. Sie entspricht der Beschreibung der Änderungen. Es ist möglich, einen Textblock z. B. über einen Texteditor einzufügen, ohne die Option `-m` zu verwenden.

Ihre Änderungen werden nun in Ihrem lokalen Git-Repo veröffentlicht. Bravo!

## 3.8 Mehr informationen

Welche Informationen erhalten Sie nun mit dem Befehl:

```
git log --oneline
```

Erläutern Sie alle Informationen in dieser Zeile deutlich.

Schreiben Sie die Antwort auf!

Sie können den Textmodus mit der Taste `q` verlassen

## 3.9 Weitere commits

Führen Sie einen weiteren Commit durch, um eine neue (leere) `hello_world.py` Datei in Ihr Repo aufzunehmen.
Welche neuen Informationen liefert Ihnen nun der Befehl:

```
git log
```

Schreiben Sie die Antwort auf!

Beachten Sie, dass jeder Commit mit einem "Hash" oder einer "Prüfsumme" (vom Typ sha1) versehen wird. Die mit dem Befehl:

```
git log --oneline
```

angezeigten Hashes sind nur die ersten paar Zeichen dieser sogenannten "short hashes".

## 3.10 Checkout commit

Führen Sie nun einen Checkout mit dem folgenden Befehl durch und verwenden Sie dabei den "short hash" , der Ihrem ersten Commit entspricht.

```
git checkout <SHA1> -b inspectingPrev
```

Was stellen Sie nun fest, wenn Sie den Inhalt des Working Directory aufrufen?

Schreiben Sie die Antwort auf!

## 3.11 Checkout master

Führen Sie nun einen Checkout mit dem folgenden Befehl durch:

```
git checkout master
```

Was stellen Sie nun fest, wenn Sie den Inhalt des working directory betrachen?

Schreiben Sie die Antwort auf!

# 4 Branch und Merge

Bisher haben wir die grundlegenden Funktionen von git verwendet. Es gibt auch die Funktionen branch (Zweig) und merge (zusammenführen), die Git im Vergleich zu den früher existierenden Tools stark vereinfacht hat.
Für diese praktische Arbeit können Sie sich mit dem GUI Sublime Merge behelfen, der Ihnen eine grafische Darstellung und einen visuellen Verlauf der Commits in Ihrem Repo liefert.

1. Erstellen Sie ein Github Repo und fügen Sie diesen als Remote hinzu (mit Kommandozeile oder GUI)
   ```
   git remote add origin https://github.com/<username>/<reponame>.git
   ```
2. Erstellen Sie in Ihrem lokalen Repo einen Entwicklungszweig `dev01`.
3. Erstellen Sie auf diesem Zweig zwei Commits:
   - Einen, um eine Datei `helloworld.py` zu erstellen (dieser Commit existiert bereits).
   - Eines, um diese Datei `helloworld.py` zu füllen.

   ```
   print("Hello, world!")
   ```
4. Checkout des `master` Zweiges
5. Vom master Branch aus erstellen Sie einen neuen Entwicklungszweig `dev02`.
6. Bearbeiten Sie die Datei `README.md` und erstellen Sie einen commit auf dem Zweig `dev02`.
7. Mergen Sie den Zweig `dev02` in `master`.
   - Checken Sie den `master`-Zweig aus
   - Führen Sie das merge „merge `dev02` into `master`“ durch.
8. Zusammenführen des Zweigs `dev01` in `master`.
   - Checken Sie den `master`-Zweig aus
   - Führen Sie das merge „merge `dev01` into `master`“ durch.
9. Pushen Sie Ihr lokales Repository auf Ihr GitHub-Remote.
   ```
   git push origin master.
   ```

Übersetzt mit www.DeepL.com/Translator (kostenlose Version)

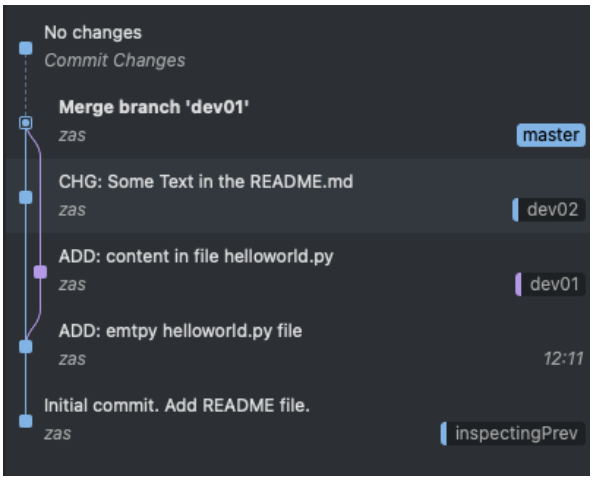


Abbildung 7: Status nach der Zusammenführung der Repository-Zweige `dev01` und `dev02`

# 5 Gitflow

Verwenden Sie für diese Aufgabe die im Kurs vorgestellte Gitflow-Philosophie.
Sie alle werden an dem folgenden Git-Repo zusammenarbeiten, als ob Sie ein Entwicklungsteam bilden würden:
https://github.com/tschinz/gitflow [5]
Dies ist ein öffentliches Git-Repo, das auf Github gehostet wird.

## 5.1 Fork

Aus Sicherheitsgründen ist es Ihnen nicht gestattet, direkt an diesem Repository zu arbeiten. Sie müssen Ihre eigene Kopie (fork) erstellen, um Änderungen vornehmen zu können. Bitte erstellen Sie daher in Ihrem GitHub-Konto einen "Fork" dieses Repositoriums. Verwenden Sie dazu die Schaltfläche "Fork" in der Weboberfläche von Github.

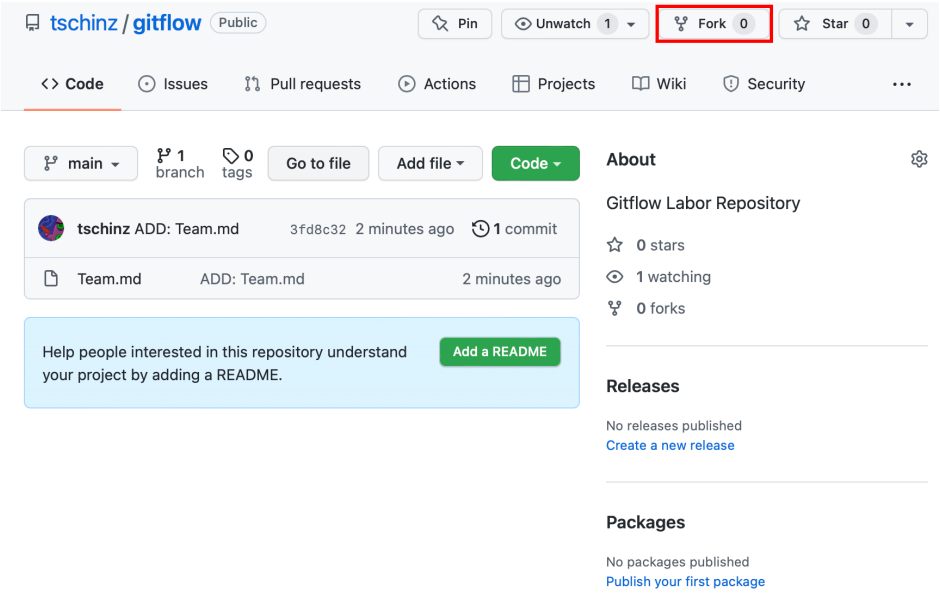


Abbildung 8: Fork Knopf für ein GitHub Repository

Klonen Sie anschliessend unser gemeinsames Repository. Die URL Ihres Repository wird ähnlich aussehen wie:

```
git clone https://github.com/<username>/gitflow.git
```

## 5.2 Parallele Zusammenarbeit

Bearbeiten Sie in einem lokalen Feature-Zweig die Datei `Team.md`. Ersetzen Sie Ihre gegebene Nummer durch Ihren Vornamen und Namen.

> ⚠ Sie werden alle die gleiche Datei bearbeiten. Um Konflikte zu vermeiden, bearbeiten Sie nur die Zeile, die für Sie relevant ist.

"Commiten" und "Pushen" Sie Ihren Zweig in das Fork-Repository auf GitHub.

## 5.3 Pull Request

Erstellen Sie einen “Pull Request” (Merge-Anfrage) auf GitHub. Verwenden Sie dazu die Schnittstelle der GitHub-Website.
Sobald alle Pull Requests fertig sind, werden die Merges in Absprache mit der gesamten Gruppe (und den Lehrern) durchgeführt.

# 6 GIT Befehle

Github git cheatsheet [6] [7]

## 6.1 Änderungen überprüfen und eine Commit-Transaktion anfertigen

```
git status
```

Listet alle zum Commit bereiten neuen oder geänderten Dateien auf.

```
git diff
```

Zeigt noch nicht indizierte Dateiänderungen an.

```
git add [file]
```

Indiziert den derzeitigen Stand der Datei für die Versionierung.

```
git diff --staged
```

Zeigt die Unterschiede zwischen dem Index ("staging area") und der aktuellen Dateiversion.

```
git reset [file]
```

Nimmt die Datei vom Index, erhält jedoch ihren Inhalt.

```
git commit -m "[descriptive message]"
```

Nimmt alle derzeit indizierten Dateien permanent in die Versionshistorie auf.

## 6.2 Änderungen synchronisieren

Registrieren eines externen Repositories (URL) und Tauschen der Repository-Historie.

```
git fetch [remote]
```

Lädt die gesamte Historie eines externen Repositories herunter.

```
git merge [remote]/[branch]
```

Integriert den externen Branch in den aktuell lokal ausgecheckten Branch.

```
git push [remote] [branch]
```

Pusht alle Commits auf dem lokalen Branch zu GitHub.

```
git pull
```

Pullt die Historie vom externen Repository und integriert die Änderungen.

## 6.3 Meistgebrauchten Git Befehle

### 6.3.1 Start a working area

- `clone` - Clone a repository into a new directory
- `init` - Create an empty Git repository or reinitialize an existing one

### 6.3.2 Work on the current change

- `add` - Add file contents to the index
- `mv` - Move or rename a file, a directory, or a symlink
- `reset` - Reset current HEAD to the specified state
- `rm` - Remove files from the working tree and from the index

### 6.3.3 Examine the history and state

- `log` - Show commit logs
- `show` - Show various types of objects
- `status` - Show the working tree status

### 6.3.4 Grow, mark and tweak your common history

- `branch` - List, create, or delete branches
- `checkout` - Switch branches or restore working tree files
- `commit` - Record changes to the repository
- `diff` - Show changes between commits, commit and working tree, etc
- `merge` - Join two or more development histories together
- `rebase` - Reapply commits on top of another base tip
- `tag` - Create, list, delete or verify a tag object signed with GPG

### 6.3.5 Collaborate

- `fetch` - Download objects and refs from another repository
- `pull` - Fetch from and integrate with another repository or a local branch
- `push` - Update remote refs along with associated objects

# Bibliographie


[1] T. Linus, "Git." https://git-scm.com/ (accessed: Apr. 25, 2023).

[2] "GitHub." https://github.com/ (accessed: Apr. 25, 2023).

[3] "GitLab Hevs," GitLab. https://gitlab.hevs.ch/ (accessed: Apr. 25, 2023).

[4] "Sublime Merge - Git client from the makers of Sublime Text." https://www.sublimemerge.com/ (accessed: Apr. 25, 2023).

[5] tschinz, "Tschinz/gitflow," 2022. Accessed: Apr. 25, 2023. [Online]. Available: https://github.com/tschinz/gitflow

[6] gitlab, "Git Cheatsheet," 2023.

[7] "GitHub Git Spickzettel," GitHub Cheatsheets. https://training.github.com/downloads/de/github-git-cheat-sheet/ (accessed: Apr. 25, 2023).